Deep Reinforcement Learning-Based Dual-Timescale Service Caching and Computation Offloading for Multi-UAV Assisted MEC Systems

Na Lin, Xiao Han, Ammar Hawbani, *Member, IEEE*, Yunhe Sun, *Student Member, IEEE*, Yunchong Guan, and Liang Zhao, *Member, IEEE*

***Abstract*—The emergence of unmanned aerial vehicles (UAVs) ushers in a new era for mobile edge computing (MEC), significantly expanding its range of service and potential applications. Due to the limited storage capacity and energy budget of UAVs, it is crucial to determine a reasonable service caching and task offloading strategy. Service caching means that task-related programs and the associated databases are cached on edge servers. In this paper, we consider the time latency and energy consumption caused by frequent changes to the service caching, aiming to jointly optimize the computational offloading, resource allocation, and service caching in multi-UAV assisted MEC systems at different time scales. The objective of this optimization is to reduce the overall system delay while staying within the energy limitations of both the UAVs and ground devices. An improved service caching policy (SCP) is proposed, which is based on task popularity and utilizes the greedy dual size frequency (GDSF) algorithm. The SCP is combined with the twin delayed deep deterministic policy gradient (TD3) algorithm to propose an innovative dual timescale TD3 (DTTD3) algorithm. The numerical outcomes obtained from a substantial number of simulation experiments demonstrate that DTTD3 outperforms existing benchmark methods in terms of convergence and parameter optimization.**

***Index Terms*—Mobile edge computing(MEC), unmanned aerial vehicles (UAVs), service caching, task offloading, resource allocation, deep reinforcement learning.**

I. Introduction

THE rapid advancement of Internet of Things (IoT) technology creates an increasing demand for ultra-low latency and better reliability in a wide range of developing mobile applications [1]. Nevertheless, IoT devices generally have restricted energy, storage and computational capabilities, which presents a significant obstacle in fulfilling the demands of compute-intensive and low-latency applications [2]. In this context, mobile edge computing (MEC) emerges as a potentially viable resolution [3]. By positioning storage and computational resources closer to the devices, MEC essentially expands the functionalities of cloud computing to the network's edge. This approach eventually results in lower latency and higher service quality. However, the deployment cost associated with infrastructure-based MEC servers is substantial, making it challenging to offer services in remote or disaster-stricken areas lacking adequate infrastructure. Unmanned aerial vehicles (UAVs) become increasingly prevalent in MEC systems as a means to overcome this constraint [4]. UAVs can enhance service quality and coverage in resource-constrained areas because of their mobility and flexibility [5]. In contrast to traditional MEC system, UAV-assisted MEC system offers greater flexibility and on-demand service delivery. UAVs operate as MEC servers by utilizing high-quality line-of-sight (LoS) communication connections, which facilitate quicker responses to user requests [6].

UAVs equipped with MEC servers can provide communication, computation, and caching services for ground devices. By intelligently offloading computational tasks to UAVs, it is possible to enhance the endurance of ground terminals while simultaneously improving the quality of service. However, current research on computation offloading often assumes that MEC servers can handle computation tasks of any type generated by devices. If the MEC server can provide various services requested by users (such as image recognition, object detection, and voice translation), then it can provide the highest quality of service. Nevertheless, compared to ground MEC servers, the cache resources of UAVs are extremely limited, and caching all services on them is practically unfeasible. Based on the above considerations, it is advisable to formulate a reasonable service caching strategy. This strategy will affect the types of tasks that can be offloaded to UAVs, thereby influencing the system's performance. Thus, it is crucial to optimize computation offloading and service caching simultaneously in a UAV-assisted MEC network.

In addition, the merger of reinforcement learning and deep learning has resulted in the emergence of deep reinforcement learning (DRL) with the progression of artificial intelligence.

Received 29 February 2024; revised 19 June 2024; accepted 23 September 2024. Date of publication 30 September 2024; date of current version 14 March 2025. This work was supported in part by the National Natural Science Foundation of China under Program 62372310, National Natural Science Foundation of China-Youth Science Foundation Program 62303331, 61902261, Liaoning Provincial Department of Education Project - Key Project LJKZ0206 and Liaoning Provincial Science and Technology Department Project - Key R&D Program 2023JH2/101300194. The associate editor coordinating the review of this article and approving it for publication was N. Kamiyama. *(Corresponding authors: Liang Zhao; Ammar Hawbani.)*

The authors are with the School of Computer Science, Shenyang Aerospace University, Shenyang 110136, China (e-mail: linna@sau.edu.cn; 18604292590@163.com; anmande@ustc.edu.cn; sunyunhe@sau.edu.cn; y.c.guan@foxmail.com; lzhao@sau.edu.cn).

Digital Object Identifier 10.1109/TNSM.2024.3468312

1932-4537 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

The DRL is recognized as a highly effective method for attaining asymptotically optimal solutions in dynamic contexts [7]. The authors in [8] present two DRL methods that take into account user assignment, bandwidth, computing resource allocation, UAV path planning and average aggregated QoE over all IoTD total time slots. For multi-UAVs cooperative target search, Luo et al. [9] suggest a deep Q-network (DQN)-based DRL architecture with separated Q-networks to determine the best choices for computing offload and flight direction. In [10], the authors jointly optimize DRL-based service function deployment, dependent task scheduling, and cooperative UAV trajectory planning. MEC environments often face changing network conditions and user requirements. Herein, DRL emerges as a critical tool, empowering the system to accommodate these frequent changes. Through autonomous policy and decision adaptations, the system can ensure an optimal response in varied situations.

In this paper, the co-optimization issue of service caching, computation offloading, and resource allocation in UAV-assisted MEC systems is investigated. In such a scenario, the conventional BS remains at a distance from the ground infrastructure, while a compact edge server embedded inside the UAV performs crucial tasks such as relaying, caching, and computation services for ground-based users. Given the inherent limitations of the battery capacity of ground devices and UAVs, our endeavor focuses on co-optimization of service caching, computation offloading and resource allocation proportions across different time scales, with the overall objective of reducing system service latency. This distinction in time scale provides a reduction in wireless backhaul time and energy consumption when the UAV caches the programs required for tasks. The primary contributions of this paper can be succinctly described as follows:

- We propose a task offloading and service caching optimization algorithm at different time scales for a MEC network assisted by multiple UAVs. Within the energy budgets of the UAVs and ground devices, service caching, computational offloading, and resource allocation are jointly optimized to satisfy the dynamic needs of the user's tasks while minimizing the overall system latency.
- We propose a service caching policy (SCP) improved by the greedy dual size frequency (GDSF) algorithm. Over a longer time interval, the service cache is updated considering task popularity, task access frequency and the size of the program required for task execution.
- We propose a dual timescale TD3 (DTTD3) algorithm, incorporating SCP into the twin delayed deep deterministic policy gradient (TD3) algorithm. Based on the service caching decision, we optimize the offloading decision and resource allocation at a smaller time scale. Simulation results demonstrate the superior performance of the DTTD3 algorithm in terms of convergence speed and total system delay.

The subsequent sections of this paper are structured as follows. Section II reviews the related works. The presentation of the system model commences in Section III. The formulation of the optimization problem is provided in Section IV. Section V describes the MDP modeling and the suggested collaborative service caching and computation offloading algorithm. Section VI displays the results of the simulations. Section VII provides the concluding remarks of this paper.

II. Related Work

Recently, there are numerous studies devote to enhancing the efficacy of UAV-assisted MEC networks. In [11], the authors define the energy-efficient scheduling problem as the joint optimization of computing and transmitting resource allocations, UAV trajectories, and task associations with the objective of reducing the energy consumption of all mobile devices. The authors of [12] aim to reduce the cruise duration of the UAV by optimizing task scheduling, user admission policy, UAV exploration trajectory, travel time, and allocations of computation and communication resources. Ning et al. [13] investigate a UAV-enabled MEC network, taking into account edge server deployment as well as multi-user computation offloading to reduce the overall computational cost of the system in a dynamic environment. In [14], a proposal for a compute offloading framework that utilizes UAVs to provide value-added edge computing services while in flight is made.

Performing a task necessitates the user's task data as input, as well as the relevant programs for processing it. The act of preserving the input and/or output data pertaining to a computing task within the confines of a server is referred to as computing content caching. For mixed augmented reality (AR) and standard multimedia applications supported by UAV base stations, cache-enabled UAV non-orthogonal multiple access (NOMA) networks are examined in [15]. The authors of [16] optimize the minimal throughput of users serviced by UAVs by effectively managing cache placement, UAV trajectory, and transmission capacity within a given time frame. Content caching reduces repetitive data transmission and task computation, which thereby reduces latency and energy consumption. Nevertheless, its effectiveness contingents upon the assumption that the cached input/output data from computing tasks will be frequently reused in future executions [17].

Correspondingly, the practice of retaining program data employed in the execution of computational tasks within the server infrastructure is denoted as service caching [18]. In consideration of latency and resource limitations, Chen et al. optimize computational offloading, resource allocation, and service cache replacement jointly within the MEC system to reduce the overall cost of all mobile devices [19]. The authors in [20] provide an optimization approach for jointly optimizing caching, offloading, and time allocation policies. The objective is to reduce the total energy usage while considering caching and deadline limitations. In [21], the authors investigate cooperative service caching and workload scheduling within MEC, emphasizing the significance of collaboration among edge nodes. The methods by which dependent tasks can be efficiently offloaded to peripheral nodes with limited service caches are examined in [22]. The authors in [23] investigate the joint optimization of task offloading and service caching in a MEC system supported by multiple UAVs.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

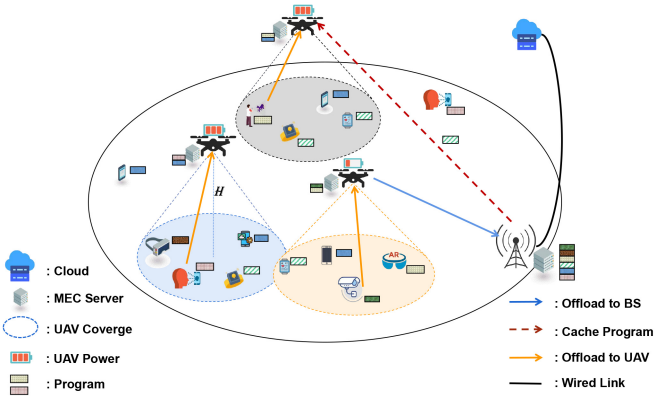


Fig. 1. Multi-UAV enabled MEC system.

The aforementioned studies on service caching are primarily focused on terrestrial MEC networks. However, investigations into the holistic optimization of communication, caching, and computation in UAV enabled-MEC networks primarily concentrate on the caching of computational content. This involves caching task input data and/or the resultant output on edge servers equipped by UAVs. Nevertheless, real-world situations provide challenges because an application's input data and calculation outputs rarely coincide precisely, lowering the likelihood of frequent reuse. Furthermore, when compared to MEC servers deployed on the ground, UAVs have limited energy and storage capacities, which hinders their capability to cache significant task data. Frequent service cache updates could introduce undesirable latency and increase energy consumption, potentially rendering the cache worthless. Therefore, considering the timescale issue of service caching and computation offloading, optimization for computation offloading should occur at a smaller timescale, while service caching should be updated at a larger timescale. In summary, it is crucial to ensure efficient coordination of joint service caching, offloading decision, and resource allocation in UAV-enabled MEC systems in order to satisfy the service quality demands of varied users.

III. System Model

The system model is depicted in Fig. 1, contains a multi-UAV enabled MEC system designed to provide services to ground devices. Our system model consists of three layers: a BS, U UAVs and M ground devices. In a post-earthquake scenario, the BS is located far from the ground devices, resulting in longer propagation delay. Additionally, the BS experiences longer queuing delay due to a high load of tasks. On the other hand, the UAVs, located closer to the ground devices, providing shorter propagation and queuing delays and can efficiently provide relay and computational services to support the ground devices. The UAVs and devices are represented by $\mathcal{U} = \{1, \ldots, U\}$ and $\mathcal{M} = \{1, \ldots, M\}$, respectively. The UAVs are integrated with small edge servers, equipped with caching and computing capabilities. The temporal domain is partitioned into T time slots, which are represented by $\mathcal{T} = \{1, \ldots, T\}$. Ground devices generate a computing task in every time slot t, where each task requires processing through a designated program. Let K indicates the number of task categories, and $\mathcal{K} = \{1, \ldots, k, \ldots, K\}$ represents the set of programs necessary to accomplish all tasks. l_k denotes the size of each program. It is assumed that the types of computational tasks generated by the ground devices obey the Zipf distribution and that one computational task is generated in each time slot while moving randomly in the next time slot. The computational tasks generated by the device m are represented by $J_m = \{L_m, W_m, K_m\}$. Here, L_m signifies the magnitude of the uploaded data pertaining to the task, W_m represents the quantity of CPU cycles necessary for the task, and K_m denotes the program essential for the computational task. High-performance edge servers are equipped by the BS, which enable it to cache all the necessary programs from cloud servers via wired connections. In contrast, UAVs have limited caching capacity, which restricts their ability to cache programs. Each UAV operates at a consistent altitude H, and is equipped with a fixed-beamwidth directional antenna characterized by θ. The service coverage area of each UAV is defined as $H \tan\theta$, and it can cater to only one ground device at a time. When the ground device is inside the UAV's service range, the UAV caches the necessary program to execute the computation task of the ground device. If the required program for the task is not cached, the UAV can transmit the task's upload data to the BS. The UAV relays the results obtained from the BS's computation of the mission to the ground device. Communication between UAVs and ground devices is performed using Time Division Multiple Access (TDMA), which ensures that each UAV can communicate with only one ground device in a time slot, thus avoiding interference caused by simultaneous transmissions. We plan the UAVs trajectory in such a way that it covers as many different devices as possible, which aims to cover a larger area and reduce the number of cases where multiple UAVs cover the same device. In the event that multiple UAVs cover the same device at the same time, the device selects one UAV for communication and task offloading based on the strength of each UAV's signal (which is usually determined by the distance), thereby ignoring interference between UAVs' communications. Table I provides a list of the primary notations along with their corresponding descriptions.

A. Communication Model

The positions of devices and UAVs are represented using the 3D Cartesian coordinate system. Since the UAVs are higher than any of the devices, the heights of the devices can be regarded as insignificant. Assume that the horizontal coordinate of device m and UAV u are represented by $q_m = (x_m, y_m)$ and $q_u = (x_u, y_u)$, respectively. Moreover, a situation is examined that is almost static, wherein the devices and UAVs stay stationary for the duration of the task offloading phase, but they may move at a later time [24]. The UAV u and the device m are separated by a Euclidean distance as in (1).

$$d_{m,u} = \sqrt{(x_m - x_u)^2 + (y_m - y_u)^2 + H^2} \tag{1}$$

It is assumed that the radio connection between the UAV and the BS as well as the forward radio connection from the device

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

TABLE I
NOTATIONS AND DESCRIPTIONS

Notation	Description
U	Number of UAVs
M	Number of ground devices
$\mathcal{T}$	The set of time slots
$\mathcal{K}$	The set of all programs
l_k	The size of program
L_m	The magnitude of the uploaded data pertaining to the task
W_m	The quantity of CPU cycles necessary for the task
K_m	The program essential for the computational task
q_m	The device m's coordinate horizontally
q_u	The UAV u's coordinate horizontally
H	Flight height of UAVs
θ	Fixed beamwidth directional antenna for UAV
R_b	The link capacity from each UAV to the base station
h_0	The channel gain at reference distance 1m
B	The total bandwidth
N_0	The noise's power spectral density at the receiver
P_m	The transmit power of device m
P_u	The transmission power of UAV u
$c_{k,u}$	The choice to cache program k at UAV u
$a_{m,u}$	The unloading decision and user association metrics for the devices
$b_{m,u}$	The allocation of computation resources from UAV u to device m as a percentage
f_m	The computing ability of device m
F_u	The total capacity for computing of UAV u

to the UAV function on separate frequency bands, successfully circumventing significant interference between both of them. Considering the UAVs' flight altitude and mobility, it is justifiable to presume that the wireless connection between the device and the UAV is predominantly influenced by the LoS link [25]. Descriptive of the channel gain between the device *m* and the UAV *u* is given by (2), where h_0 represents the channel gain at the reference distance $d = 1$ m.

$$h_{m,u} = \frac{h_0}{d_{m,u}^2} \tag{2}$$

Therefore, the uplink data rate between device *m* and UAV *u* is given by (3), where P_m signifies the transmit power of device *m*, N_0 designates the power spectral density of the noise at the receiver, and *B* denotes the overall bandwidth.

$$R_{m,u} = Blog_2\left(1 + \frac{P_m h_{m,u}}{BN_0}\right) \tag{3}$$

B. Caching Model

To reduce computational latency, we consider caching the programs needed for task computation in the UAV. Given the UAV's constrained caching capacity and the inability to cache all programs simultaneously, judicious deliberation is required regarding which programs warrant caching. Furthermore, due to the high communication latency between the BS and the UAV, cache decision is modified every T' time slots. The binary variable $c_{k,u}$, given by (4), is defined to indicate the caching decision for program *k* at UAV *u*; $c_{k,u} = 1$ represents that the program *k* is cached on UAV *u*, and $c_{k,u} = 0$ signifies that UAV *u* does not cache program *k*.

$$c_{k,u} = \{0, 1\}, \forall k \in \mathcal{K}, u \in \mathcal{U} \tag{4}$$

The following constraint (5) is imposed as a result of the restricted cache capacity of each UAV, where C_u represents the overall caching capacity of UAV *u*.

$$\sum_{k \in \mathcal{K}} c_{k,u} l_k \leq C_u, \forall u \in \mathcal{U} \tag{5}$$

According to the above equation, the cumulative number of cached programs remains below the maximum capacity for caching of each individual UAV.

C. Computing Model

In each time slot *t*, a computational task J_m is generated by the device *m*. Tasks strictly follow the binary offloading policy, which permits local execution, offloading to a UAV, or execution at the BS. Therefore, a variable $a_{m,u}$ is defined as the offloading decision for the task J_m as in (6), where $\mathcal{U}^o = \{0, 1, \ldots, U, U+1\}$.

$$a_{m,u}(k) = \{0, 1\} \quad k \in \mathcal{K}, m \in \mathcal{M}, u \in \mathcal{U}^o \tag{6}$$

When $a_{m,u}(k) = 0$, it signifies that the task J_m of the device *m* is performed locally. $a_{m,u}(k) = 1$ indicates that the task J_m is offloaded for execution. Moreover, $\mathcal{U}^o$ is the set that denotes the task computation location and the user association indicator. If the ground device is within the coverage of multiple UAVs, priority is given to being associated with the closest UAV. $a_{m,u}(k) = 1(u \in \mathcal{U})$ indicates task J_m is offloaded to UAV *u* for computation. $a_{m,u}(k) = 1(u = U+1)$ denotes that the task J_m is ultimately computed by the BS. $a_{m,u}(k) = 1(u = 0)$ signifies that the task is computed locally by the device *m*.

1) Computing at Devices: Each device has the programs needed to accomplish the computational tasks. Therefore, if the device executes the task locally, the task computation latency can be expressed as by (7), where f_m indicates the computing ability of device *m*.

$$T_m^{local} = \frac{W_m}{f_m} \tag{7}$$

2) Offloading Tasks to UAVs: If the UAV *u* is associated with device *m* and cached the required program for task J_m execution, the task can be uploaded to UAV *u* for computation. The device *m* uploads the task data to UAV *u*, and the time for the task's upload data L_m to be transmitted to UAV *u* is given by (8).

$$t_{m,u}^{trans} = \frac{L_m}{R_{m,u}} \tag{8}$$

Designate F_u as the overall computational capacity of UAV *u*. The time required for UAV *u* to complete the task L_m generated by device *m* is given by (9), where $b_{m,u} \in$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

$[0, 1]$ represents the proportion of the computational resources assigned to device m on UAV u.

$$t_{m,u}^{com} = \frac{W_m}{b_{m,u} F_u} \tag{9}$$

3) Offloading Tasks to BS: If the program required for task J_m is not cached on the UAV u, the data uploaded for that task is transmitted to the BS. For the sake of simplicity, the link capacity between each UAV and the BS is presumed to be R_b. The transmission time to the BS is given by (10).

$$t_{m,u}^{BS} = \frac{L_m}{R_b} \tag{10}$$

The transmission time of the computing results from UAV u to device m is considered insignificant as a result of the limited amount of data included within the results [26]. Considering the strong computational power and energy availability of the BS, the task execution time is considered to be approximated to 0. Consequently, the aggregate delay for the task to be offloaded from device m for execution during time slot t is given by (11).

$$T_{m,u}^{offload} = t_{m,u}^{trans} + \sum_{u=1}^{U} a_{m,u}(k) t_{m,u}^{com} + a_{m,U+1}(k) t_{m,u}^{BS}, \forall k \in \mathcal{K}, u \in \mathcal{U} \tag{11}$$

D. Energy Consumption Model

1) Computing at Devices: The computing energy consumption is the only component of the energy consumption of the task performed on device m. The computing task results in an energy consumption of the device m is given by (12), where δ is a correlation coefficient [27].

$$E_m^{local} = \delta f_m^2 W_m \tag{12}$$

2) Offloading Tasks: When device m transfers the task to UAV u for execution, the device's energy consumption for that task upload is:

$$E_{m,u}^{trans} = P_m t_{m,u}^{trans} \tag{13}$$

If the UAV u has cached the necessary program for the device's task, the calculation for the energy expenditure of the UAV u performing mission J_m is as follows:

$$E_{m,u}^{com} = \delta \left(b_{m,u} F_u\right)^2 W_m \tag{14}$$

If the UAV u unloads the task to BS for execution, the energy expenditure of UAV u transmission task is presented as in (15), where P_u is the transmission power of UAV u.

$$E_{m,u}^{BS} = P_u t_{m,u}^{BS} \tag{15}$$

IV. Problem Formulation

In this section, we provide the formulation of the optimization problem with the objective of minimizing the total task delay. This optimization takes into account the energy constraints imposed on both the devices and UAVs. The task completion latency of device m is given by (16). The amount of energy required for device m to finish task is formulated as (17), while the energy consumption of UAV u to complete task is expressed as in (18).

$$T_m = a_{m,0}(k) T_m^{local} + \sum_{u \in \mathcal{U}^o} a_{m,u}(k) T_{m,u}^{offload} \tag{16}$$

$$E_m = a_{m,0}(k) E_m^{local} + \sum_{u \in \mathcal{U}^o} a_{m,u}(k) E_{m,u}^{trans} \tag{17}$$

$$E_u = \sum_{u=1}^{U} a_{m,u}(k) E_{m,u}^{com} + a_{m,U+1}(k) E_{m,u}^{BS} \tag{18}$$

The objective of our system is to minimize the total system delay as in (19a) while staying within the energy consumption restrictions of all ground devices and UAVs. This is accomplished through the collaborative optimization of service cache policy $(\mathcal{C}(t))$, offloading decision $(\mathcal{A}(t))$, and computing resource allocation $(\mathcal{B}(t))$.

$$\min_{\mathcal{C}(t), \mathcal{A}(t), \mathcal{B}(t)} \quad \max_{m \in \mathcal{M}} T_m, \tag{19a}$$

$$s.t. \quad a_{m,u}(k) \in \{0, 1\}, \forall m \in \mathcal{M}, u \in \mathcal{U}^o, \tag{19b}$$

$$c_{k,u} \in \{0, 1\}, \forall k \in \mathcal{K}, u \in \mathcal{U}, \tag{19c}$$

$$b_{m,u} \in [0, 1], \forall m \in \mathcal{M}, u \in \mathcal{U}, \tag{19d}$$

$$E_m \le E_m^{max}, \forall m \in \mathcal{M}, \tag{19e}$$

$$\sum_{u \in \mathcal{U}} a_{m,u}(k) E_u \le E_u^{max}, \forall m \in \mathcal{M}, \tag{19f}$$

$$\sum_{u \in \mathcal{U}^o} a_{m,u} = 1, \forall m \in \mathcal{M}, \tag{19g}$$

$$\sum_{u \in \mathcal{U}} a_{m,u}(k) b_{m,u} \le 1, \forall m \in \mathcal{M}, k \in \mathcal{K}, \tag{19h}$$

$$\sum_{k \in \mathcal{K}} c_{k,u} l_k \le C_u, \forall u \in \mathcal{U}, \tag{19i}$$

$$a_{m,u} d_{m,u} \le H \tan\theta, \forall m \in \mathcal{M}, u \in \mathcal{U} \tag{19j}$$

The binary variables that denote the service caching and offloading decision are represented by Constraint (19b) and Constraint (19c), respectively. Furthermore, the computational resource allocation variable, which is a continuous variable, is indicated by constraint (19d). Constraint (19e) signifies that the energy consumed by each device to complete its task must not surpass its predetermined energy budget, denoted as E_m^{max}. Constraint (19f) establishes a restriction on the energy usage of every UAV during the execution of offloading device-related tasks, guaranteeing that it does not surpass its specified energy budget, denoted as E_u^{max}. Constraint (19g) stipulates that each device is permitted to select a single location for the execution of its task. Constraint (19h) implies that the total computational resources allocated to all offloading devices must not exceed the computing capacity of each UAV. Constraint (19i) guarantees that the cumulative number of cached programs remains within the caching capacity of each UAV. As per Constraint (19j), it is imperative that the positioning of the offloading devices aligns with the coverage area of the UAV.

To determine the most effective approach for compute offloading, service caching, and resource allocation, it is imperative for the system to collect extensive data and make

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

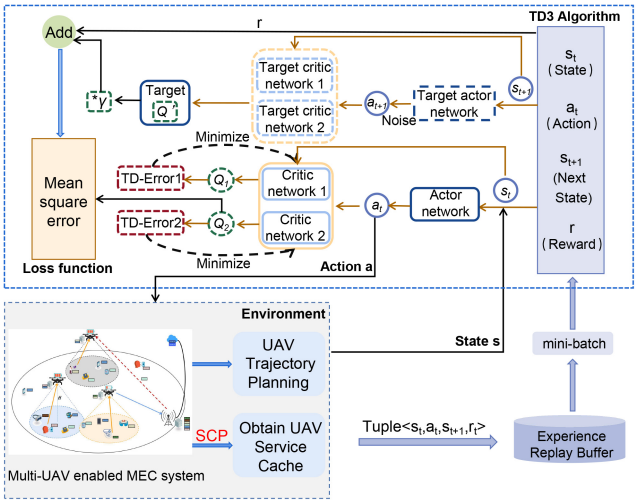


Fig. 2. Overall framework of the system.

informed decisions. Additionally, the objective of optimization is a mixed integer nonlinear programming (MINLP) problem. Therefore, we propose an algorithm based on DRL to address this optimization challenge in the complex context.

V. Proposed DRL-Based Algorithm

The optimization problem is represented in this section as an MDP, complete with a clearly defined state space, action space, and reward function. Building upon these foundational efforts, we implement a DRL-based dual timescale twin delayed deep deterministic policy gradient (DTTD3) algorithm in order to mitigate cumulative latency in highly dynamic environments.

A. Markov Decision Process

The MDP as a foundational mathematical framework within the domain of reinforcement learning, furnishing a systematic model for this learning paradigm. A MDP consists of the following elements: $\{S, A, R, P, \gamma\}$. *S* represents the state space, which is a comprehensive set that includes all possible states. *A* denotes the action space, which comprises all potential actions. *P* stands for the state transition function, which quantifies the likelihood of transitioning from the current state and action to a new state. The state transition function typically represented as $P(s_{t+1}, s_t|a_t)$, indicates the probability of arriving at the new state s_{t+1} after being in state s_t and executing action a_t. In the interim, reward function *R* specifies the instantaneous incentive obtained by an agent upon performing an action and changing states. The reward function is commonly represented as $R_t = r(s_t, a_t)$. This function embodies the reward acquired upon doing action a_t in state s_t, resulting in the transition to state s_{t+1}. The discount factor γ is a numeric value between 0 and 1 that is employed in the decision-making procedure to ascertain the significance of forthcoming rewards. When the discount factor is closer to 0, the agent primarily prioritizes immediate rewards. Conversely, when the discount factor approaches 1, the agent places greater emphasis on long-term rewards.

B. Definition of the State, Action and Reward

The definition of state, action, and reward function plays a critical role in DRL, directly affecting the optimization performance. We define these elements based on the system model and optimization objective.

1) States Space (S): The environment is represented by a state space that encompasses the position of UAVs $\mathcal{L}^U(t)$ at the current time slot *t*, the UAVs' caching programs $\mathcal{C}^U(t)$ and the UAVs' remaining battery capacity $\mathcal{B}^U(t)$. Additionally, we consider the location $\mathcal{L}^d(t)$ of the ground devices, the task size generated by the ground devices $\mathcal{W}^d(t)$ and the task type $\mathcal{K}^d(t)$ generated by the ground devices. We utilize the state normalization method to minimize the size disparity between various states. This is achieved by employing the interval separating the highest and lowest values of the state as a scaling variable. Definition of the state space is: $s(t) = \{\mathcal{L}^U(t), \mathcal{C}^U(t), \mathcal{B}^U(t), \mathcal{L}^d(t), \mathcal{W}^d(t), \mathcal{K}^d(t)\}$.

2) Action Space (A): The action space consists of the ground devices that are being served, the strategy adopted for computation offloading and the computation resource allocation proportions. Within each time slot *t*, the UAV may provide service to all ground devices within its range, but only one device at a time. The agent chooses whether to offload ground device tasks or execute them locally on the device based on the current state. The task is uploaded to the UAV for execution, and the UAV needs to allocate a certain percentage of computational resources for computation. Therefore, the action space can be described as: $a(t) = \{\mathcal{X}(t), \mathcal{A}(t), \mathcal{B}(t)\}$, where $\mathcal{X}(t)$ is the ID of the ground device being served by the UAV, $\mathcal{A}(t)$ is the offloading decision, and $\mathcal{B}(t)$ is the resource allocation proportions.

3) Reward Function: The agent selects actions and continuously updates the strategy to maximize rewards by calculating expected rewards. The optimization goal is to minimize the total system latency, which is the opposite of maximizing the cumulative discount reward in DRL. Therefore, the reward function can be defined as a negative value of delay:

$$R_t = -T_m$$

C. Proposed DRL-Based Algorithm

We propose a DRL-based DTTD3 algorithm to optimize service caching, computation offloading, and resource allocation at different time scales. Fig. 2 displays the system's general architecture. First, we propose an improved service caching policy (SCP) to update the UAV's service cache on a larger time scale (every t_{cache} time slots). Then, on a smaller time scale (every time slot), we plan the trajectories of multiple UAVs with the objective of maximizing the number of different devices covered. After obtaining the state of the UAV's service cache and location, we use the TD3 algorithm to optimize the offloading decision and the resource allocation proportions.

1) Service Caching Policy: In order to optimize service caching on a larger time scale (T'), we propose an improved SCP based on the popularity of tasks and the principle of the greedy dual size frequency (GDSF) algorithm. The GDSF is a cache management algorithm designed to optimize cache

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

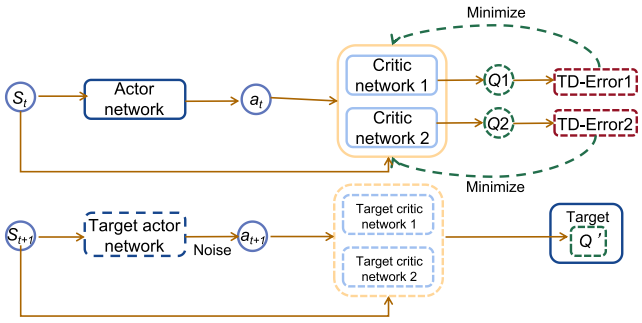


Fig. 3. The TD3's network structure.

performance by making cache replacement decisions based on the size and access frequency of data items to improve hit rates and reduce wasted cache space. The algorithm combines two key factors: the size of data items and the access frequency of data items to more effectively manage the cache.

In our SCP, the access frequency of data for each program is calculated based on the number of requests n in the past T' time slots. Additionally, the size of the data item is determined by the size of each program l_k. Meanwhile, we consider the popularity of the tasks and calculate the popularity factor for each task based on the Zipf distribution as in (20), where k represents the numbered ranking of each task. S denotes the parameter of the Zipf distribution, usually a positive real number, and C is a normalization constant that ensures that the probabilities sum to 1.

$$P(k) = \frac{C}{k^S} \tag{20}$$

To calculate the evaluation score for each program, we employ the evaluation function given by (21). Factors such as program size, popularity factor and access frequency are considered. The adjustment factors of the evaluation function, which are used to balance the influence of popularity and access frequency, are represented by β and γ.

$$S(k) = l_k\left(1 - e^{-\frac{p}{\beta}}\right)e^{-\frac{n}{\gamma}} \tag{21}$$

We establish a threshold K' for service caching, sort all programs based on evaluation scores, and then perform service caching based on the sequence of scores and program size. Specifically, we compute each program's score every T' time slots and sort each program from highest to lowest score. If multiple programs have the same score, they are sorted from smallest to largest by program size. Choose the top K' programs in terms of score ranking and cache them for the next T' time slots. If the size of the selected K' programs exceeds the total cache capacity of the UAV, the program from $K'+1$ to the last in the sequence is selected for replacement until the upper limit of the cache capacity is reached.

2) TD3 Algorithm for Computation Offloading and Resource Allocation: Within each time slot T', the UAV's caching programs are updated based on the SCP. The caching state of the UAV also plays a crucial role in determining the UAV's decision-making process regarding task offloading. During each time slot t, the offload decision and resource allocation proportions are determined by utilizing the TD3 algorithm, which is based on the current environmental conditions.

TD3 utilizes an "actor-critic" architecture that consists of six networks: two actor networks (the actor network and the target actor network), and four critic networks (two critic networks and two target critic networks). The actor network, also known as the policy network, is responsible for generating actions based on the current state to maximize the expected cumulative reward. It inputs the current state and outputs deterministic actions. Critic network is used to evaluate the value of the actions selected by the policy network. Critic network computes the corresponding Q-value by inputting the state and the action, reflecting the expected cumulative reward of taking the action from that state under the current policy. The target network consists of the target policy network and the target critic network, which are delayed update copies of the policy network and the critic network. The parameters of the target network are slowly updated through a soft update policy, and this delayed update mechanism effectively reduces the fluctuation of the target value during the training process. Fig. 3 depicts the network architecture of TD3. Within each t time slot, the agent chooses an action (a_t) through strategy (π), which depends on the current environmental state (s_t):

$$a_t = \pi(s_t) + \mathcal{N} \tag{22}$$

The agent receives a reward (r) from the environment, followed by transitioning into a new state (s_{t+1}). The generated transition (s_t, a_t, r, s_{t+1}) is preserved within the replay buffer. A batch size of data is extracted from the replay buffer during the training phase. For each transition (s_t, a_t, r, s_{t+1}) in batch, utilizing the target actor network to choose the action in state s_{t+1} is given by (23), where ϵ is a noise.

$$\tilde{a} = \pi(s_t) + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c) \tag{23}$$

TD3 introduces the concept of smoothing by incorporating noise into the action outputs of the target policy. This approach is aimed at mitigating overfitting by facilitating a smoother estimation of the Q-value function.

In reinforcement learning, when discrete actions are being learned, the value function is frequently overestimated. This happens as a result of the maximal Q-value frequently selected for evaluation during the Q-value function estimation process, which generally results in an estimate higher than the actual maximum Q-value. The cumulative overestimation may cause the algorithm to adopt a suboptimal strategy, resulting in things such as divergence in behavior. Similarly, overestimation of Q-value is presented in the Actor-Critic framework, which handles the control of successive actions. In TD3, two critic networks with the same network framework are introduced. The target value for estimating the state-action value of the subsequent state pair (s_{t+1}, a_{t+1}) is computed by using the smaller of the two values:

$$y = r + \gamma \min\left(Q_1'(s_{t+1}, \pi'(s_{t+1})), Q_2'(s_{t+1}, \pi'(s_{t+1}))\right) \tag{24}$$

The gradient descent algorithm is employed for the purpose of minimizing the error between the evaluation value and the

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

target value, thereby facilitating the update of parameters in both critic1 and critic2 networks:

$$\theta_{Q_i} = argmin_{\theta_{Q_i}} N^{-1} \sum_{t=1}^{N} (y - Q_i(s_t, a_t))^2 \tag{25}$$

Then, the policy network is updated with the deterministic policy gradient:

$$\nabla_{\theta_\pi} J \approx \frac{1}{N} \sum_{t=1}^{N} \nabla_a Q_1(s_t, a_t)|_{a=\pi(s_t)} \nabla_{\theta_\pi} \pi(s_t) \tag{26}$$

Furthermore, the TD3 algorithm incorporates a lower update frequency for the actor network compared to the critic network. This deliberate delay in updates serves a dual purpose. Firstly, it minimizes redundant and unnecessary updates. Secondly, the delayed update strategy helps in mitigating the accumulation of errors resulting from multiple updates. For target network, TD3 also adopts the soft update approach, which is stated as follows:

$$\begin{aligned} \theta_{Q_i'} &= \tau\theta_{Q_i} + (1-\tau)\theta_{Q_i'}, \\ \theta_{\pi'} &= \tau\theta_{\pi} + (1-\tau)\theta_{\pi'} \end{aligned} \tag{27}$$

here, τ represents the update coefficient, typically set to a small value, such as 0.1 or 0.01. θ_Q denotes the current network parameters of the critic, while θ_π signifies the actor's current network parameters. Additionally, $\theta_{Q_i'}$ and $\theta_{\pi'}$ correspond to the target network parameters of the critic and actor, respectively.

Algorithm 1 is our proposed DTTD3 algorithm. The DTTD3 algorithm is proposed based on the SCP and TD3 algorithm to optimize service caching, offload decision, and resource allocation proportions at different time scales. Specifically, network parameters and hyperparameters are initialized first. Then, computation offloading and resource allocation actions are executed at the beginning of each time slot t, and the service cache is updated at the beginning of each time slot T'. Process computing tasks within each time slot t based on caching, offloading, and resource allocation strategies. Subsequently, a mini batch of data is randomly sampled from the experience replay buffer and used to update the two critic networks to estimate the value of the action. Update the actor network using the policy gradient every certain number of training steps to maximize the estimation of action value by the critic network. Through soft updating the target network, the parameters of the current learning network are gradually transmitted to the target network, enhancing the training stability. This alternating training process iterates continuously throughout the training period until reaching the predetermined number of training steps or meeting task requirements.

The DRL model is constructed by deploying a central server to regularly collect state data from ground devices and UAVs, and using the large amount of data collected for centralized training. After the training is completed, the trained DRL model is deployed to each UAV, which independently senses the state of the environment and makes autonomous decisions, thus reducing communication delays and dependence on the central server.

Algorithm 1 DTTD3 Algorithm for Service Caching, Computing Offloading and Resource Allocation

```
Initialize TD3 networks parameters, episode length N_episodes and exploration noise 𝒩
Initialize replay buffer R, mini-batch size N
Initialize caching time steps T' = t_cache
for episode = 1 : N_episodes do
  Initialize episode and observe initial state s
  for t = 1 : T do
    Calculate the popularity factor for each task by (20)
    if t = t_cache then
      Calculate the evaluation score for each program by (21)
      Update cached programs
    end if
    Choose the action by (22)
    Execute action a_t, observe reward r and new state s_{t+1}
    Store transition (s_t, a_t, r, s_{t+1}) in R
    Sample a random mini-batch of transitions (s_t, a_t, r, s_{t+1}) from R
    for each transition (s_t, a_t, r, s_{t+1}) in batch do
      Compute the action in state s_{t+1} by (23)
      Compute target Q-value by (24)
      Update critic networks by (25)
      if t mod d then
        Update policy network by (26)
        Update target networks by (27)
      end if
    end for
  end for
end for
```

D. Computational Complexity

Here, we analyze the algorithm complexity of DTTD3. The SCP algorithm consists of two stages. First, iterate over all tasks and calculate their scores with computational complexity $O(n)$. Then, the task scores are sorted with computational complexity $O(nlogn)$. Thus, the overall computational complexity of the SCP algorithm is $O(Tnlogn)$, where n is the number of tasks and T is the total number of time slots. According to the analysis in [28], the complexity of offline training and online execution of DRL is mainly based on the number of matrix multiplication. For the proposed TD3 based offloading and resource allocation method, the training process runs offline and is executed in the BS with sufficient computing resources. If the actor and critic networks of each intelligent agent contain F and P fully connected layers, the complexity for training the TD3 model is $\mathcal{O}(N_{episodes}(\sum_{f=0}^{F-1} n_f n_{f+1} + \sum_{p=0}^{P-1} n_p n_{p+1}))$, where $N_{episodes}$ is the number of training episodes, and n_f represents the neuron number of layer f. The online execution computational complexity of the actor network is $\mathcal{O}(\sum_{f=0}^{F-1} n_f n_{f+1})$.

VI. Simulation Results

This section presents numerical results to evaluate the performance of our DTTD3 algorithm. Initially, we describe

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS

Parameters	Value
The number of time slots (T)	100
Number of devices (M)	20
Number of UAVs (U)	3
Height of UAVs (H)	100m
Number of task types (K)	10
The size of each program (l_k)	[0.1,0.2]Mbits
Each UAV's cache capacity (C_u)	1.0Mbits
The size of each task's input data (L_m)	[0.1,0.2]Mbits
Reference channel gain (h_0)	-50dB
Noise power density (N_0)	-169dBm/Hz
The transmission power of devices (P_m)	0.1W
Each task's CPU cycle count (W_m)	[0.2,1]Gcycles
The computing capability of each UAV (F_u)	10GHz
The capability of each device's processing (f_u)	0.5GHz
Every UAV's energy budget (E_u^{max})	300J
Energy budget of each device (E_m^{max})	[2,10]J
Total bandwidth (B)	10MHz
Wireless backhaul capacity (R_b)	1Mbps

the simulation parameters and simulation settings. Next, we analyze the impact of different network hyperparameters on the convergence of our algorithm. Finally, we evaluate our algorithm DTTD3 by comparing to benchmark algorithms under different evaluation metrics. Our source code is available via the link.[1]

A. Parameters Settings

Based on our system model, explained in Section III, we assume that the work area is 200m * 200m, with $M = 20$ devices that are deployed. We employed $U = 3$ UAVs to provide caching and computing services to the devices at a fixed height of $H = 100$m. At each time slot t, the ground devices obey the zipf distribution to generate different types of computational tasks with input data ranging from 0.1Mbits to 0.2Mbits, and then move 5m at the next time slot. The execution of each task requires a specific program. The system contains 10 different types of computational tasks, each requiring programs with cache capacity requirements ranging from 0.1Mbits to 0.2Mbits. Assuming that all UAVs have the identical energy budgets, computing capacities, and cache capacities. Each UAV has 10GHz of computation capacity and 1.0Mbits of cache capacity. The available spectrum has a channel bandwidth of 10MHz. The determination of simulation parameters follow the previous works [16], [23]. Table II provides a summary of the simulation parameters and settings.

B. The Convergence Performance

We compare the reward convergence performance of our proposed DTTD3 algorithm under different hyperparameters.

[1]https://github.com/NetworkCommunication/DTTD3

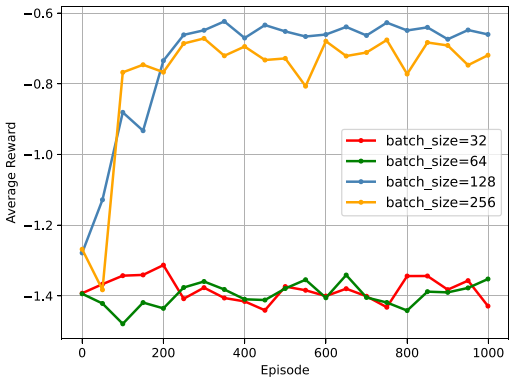


Fig. 4. The effect of different batch sizes on the performance of DTTD3.

Fig. 4 indicates the effect of different batch sizes on the algorithm's convergence performance. Batch size in reinforcement learning refers to the number of samples randomly sampled from the replay buffer at each parameter update. When we set the batch size to 32 or 64, our algorithm did not converge after 1000 episodes of training. This is because reinforcement learning algorithms typically use optimization methods such as random gradient descent to update model parameters. Gradient estimation based on a small number of samples may not be sufficient to capture the overall data distribution's characteristics. This may cause the model parameters to be updated in the opposite direction, thus preventing convergence. Our algorithm can converge when the batch size is 128 and 256. The algorithm's convergence performance is obviously higher when the batch size is 128. Therefore, we decide to set the batch size to 128.

Fig. 5 illustrates the effect of different learning rates on algorithm performance. The learning rate determines the step size used when updating the value function or strategy. After 1000 episodes of training, our algorithm still does not converge when the learning rates of the actor and critic network are 0.01 and 0.02, respectively. Because the learning rate is too large, the model parameters may skip the optimal solution in the parameter space, rendering the model incapable of converge to a suitable strategy. We set the learning rates of the actor and critic network to 0.00001 and 0.00002, respectively. Due to the lower learning rate, the algorithm converges to a local optima with low average returns. When we choose 0.0001 and 0.0002 as the learning rate of actor and critic network respectively, our algorithm converges and the reward value outperforms the reward when the learning rates are 0.00001 and 0.00002, respectively. Therefore, we set the learning rates to 0.0001 and 0.0002 for the actor network and the critic network.

C. Representative Approach for Comparison

We compared the proposed DTTD3 algorithm with different DRL algorithms to evaluate the convergence performance of our algorithm. We verified the advantage of DTTD3 over DTDDPG, DTAC and DTDQN in converging the average reward through simulation experiments.

- *DTDDPG:* This algorithm combines our proposed SCP and DDPG [19] algorithm. The offloading and resource

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

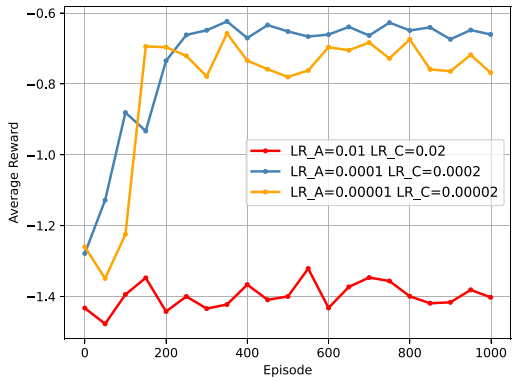


Fig. 5. The effect of different learning rates on the performance of DTTD3.

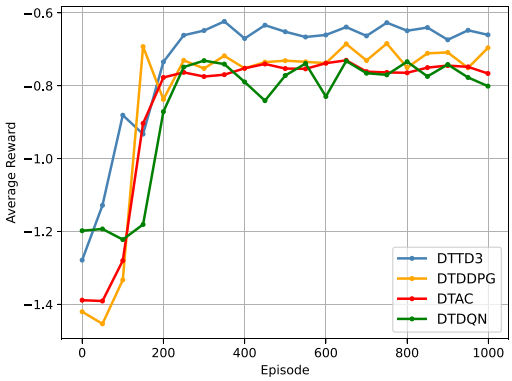


Fig. 6. Average reward of different DRL algorithms.

allocation are addressed through implementation of the DDPG algorithm, utilizing common training parameters with the TD3 algorithm.

- *DTAC:* This algorithm combines our proposed SCP and Actor-Critic (AC) algorithm. The AC algorithm solves the optimization problem associated with making decisions about computing offloading and resource allocation [29].
- *DTDQN:* This algorithm combines our proposed SCP and DQN algorithm. The DQN algorithm is used to solve the optimization problems of computing offloading [9] and resource allocation.

To evaluate the effectiveness of our proposed SCP and differentiated time-scale optimized service caching and computational offloading schemes, we compare the DTTD3 algorithm with different caching policies, including optimized service caching and computation offloading caching policies using the TD3 algorithm at the single time scale, random caching policy, and the policy in which tasks are only executed locally at the device.

- *TD3:* The service caching, offloading decision and resource allocation proportions are optimized in a single time scale and obtained by TD3 [30]. The neural network and parameters utilized are identical to those in our DTTD3 algorithm.
- *Random-Cache:* The UAV randomly caches the programs required for task execution until the cache capacity is full. The DDPG algorithm is utilized to solve offloading decisions and computational resources allocating proportion [31].
- *Local-Only:* All tasks are computed locally on the device, without any UAV-assisted computation.

D. Comparison Results

1) Average Reward of Different DRL Algorithms: Fig. 6 shows the performance of our DTTD3 algorithm compared to other DRL algorithms in terms of average reward for convergence. It can be seen that both our DTTD3 algorithm and the other benchmark algorithms start to converge after 200 episodes, but the converged average reward of the DTTD3 algorithm is better than that of the DTDDPG, DTAC and DTDQN. The DTDQN algorithm discretizes the continuous action of resource allocation, which may lead to a lack of precision in the learned policy, ultimately resulting in convergence performance inferior to other algorithms. Furthermore, our DTTD3 algorithm and DTDDPG algorithm outperform the DTAC algorithm. This is due to the one-step updating of AC and the lack of an empirical replay memory, which causes a slow convergence rate. In contrast, the DDPG-based approach enhances the effectiveness of DNN learning by using empirical replay memory and a dual network topology. In comparison to DDPG, the TD3 employs a dual-network, delayed-update technique for faster convergence and greater performance. Thus, our DTTD3 algorithm outperforms DTDDPG in terms of convergence.

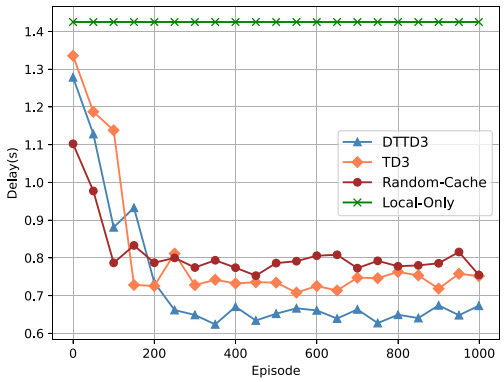


Fig. 7. Task execution delay with different cache policies.

2) Delay vs. Cache Policy: We evaluate the overall latency of computing tasks under different caching policies. As depicted in Fig. 7, our proposed DTTD3 is superior to other benchmark schemes. Unlike TD3 [30] scheme, which updates cache and offload choices at the same time scale, our DTTD3 updates them at different time scales. This method minimizes UAV energy consumption, reduces the amount of time needed for cache updates, and results in fewer service caching updates on UAVs. Although the TD3 algorithm optimizes the caching and offloading decisions at the same time scale, TD3 still outperforms the Random-Cache [31] in terms of delay when compared to the Random-Cache which is not optimized for caching. The Local-Only computing tasks are all executed by the device itself and do not rely on UAVs for computation, so the cache policy does not affect its performance.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

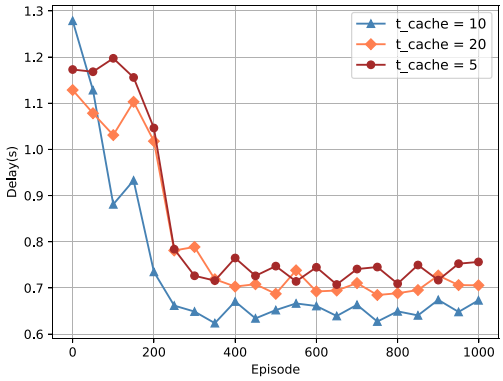


Fig. 8. Task execution delay with different t_{cache}.

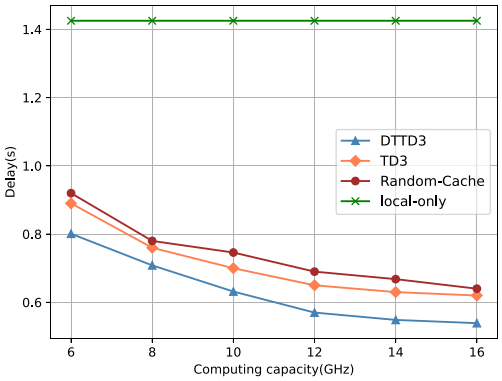


Fig. 10. Task execution delay with different computing capacities.

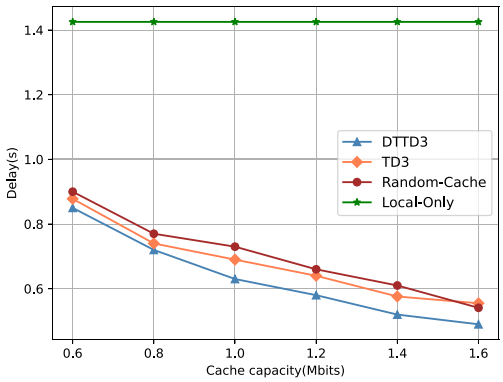


Fig. 9. Task execution delay with different cache capacities.

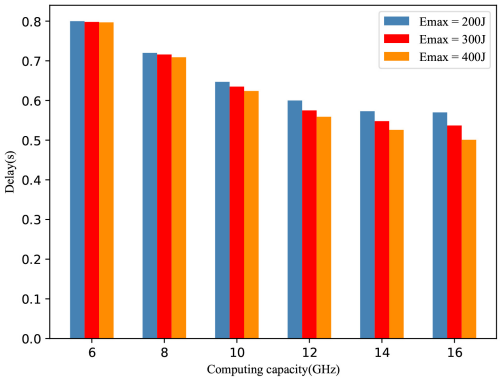


Fig. 11. The influence of maximum energy and computing capacity of UAV on time delay.

3) Delay vs. t_{cache}: The total system delay for different t_{cache} is shown in Fig. 8. When the service cache is updated too frequently, it increases system communication overhead and UAV energy consumption, reducing UAV battery life. In addition, frequent service cache updates can prevent the full utilization of the service cache benefits, as the cached content is replaced before it can be fully utilized, resulting in a meaningless timescale for distinguishing between caching and offloading. When the service cache is updated too infrequently, the devices in the UAV coverage area may change completely with the movement of the ground devices, at which time the programs cached by the UAV may not be fully utilized by the ground devices, thus reducing the overall efficiency of the system. If the value of t_{cache} is not chosen reasonably, the UAV provides more of a relay service than a computing service. Therefore, combining the experimental results, we set the value of t_{cache} to be updated once in 10 time slots.

4) Delay vs. Cache Capacity: The comparison of the delay performance of our algorithm with other benchmark methods for different UAV cache capacities is shown in Fig. 9. The total delay of the system decreases as the cache capacity increases. This is due to the fact that when the cache capacity is small, the type of tasks that can be computed by the UAV are very limited and the total system latency is all the higher. The UAV with larger cache capacity is able to cache more services, allowing the UAV to perform more types of computational tasks, thus reducing the system delay. Compared to other benchmark approaches, the performance of our DTTD3 remains superior. Furthermore, the difference in performance between our DTTD3 and other methods gets larger as cache capacity increases. This is due to our DTTD3 makes service caching decision over a longer time scale, and the larger cache capacity also reduces service caching updates, thus saving the time delay associated with frequent service caching updates. It is evident that Local-Only has consistently poorer performance as it does not rely on UAV computing tasks.

5) Delay vs. Computing Capacity: Fig. 10 presents the relationship between time delay and UAV computing capacity. The computing power of the UAV is an important factor that affects the computation time delay. As shown in the graph, when the computing capacity of the UAV is lower than 14GHZ, the overall delay of the system under different caching policies all gradually decreases as the computing capacity increases. While Local-Only tasks are performed locally, the computing capacity of the UAV has no effect on the delay. The total latency of our DTTD3 algorithm still outperforms other benchmark schemes. However, our experiments indicate that the reduction in delay becomes insignificant when the computational power F_u exceeds 14GHz. This is primarily due to that the UAV utilizes a specific amount of energy for every computational task, but the UAV's energy is restricted. When the computing capacity reaches a certain threshold (F_u = 14GHZ), the primary factor influencing the offloading of tasks to UAVs is the UAVs' energy.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

6) Energy vs. Computing Capacity: The impact of UAV computing capacity on task execution delay for different UAV energy budgets is illustrated in Fig. 11. The increase in computing capacity reduces the time required for the task to execute on the UAV, thus reducing the total system latency. However, as a result of the UAV's constrained energy budget, the reduction in task execution latency is less pronounced as the computing capacity increases. Specifically, the computational delay of the task no longer decreases once the computing capacity exceeds 14GHz while the energy budget is 200J. In addition, when computing capacity F_u is equal to 6GHz, the delays are almost equal for different energy budgets. This is because when the computing capacity is small, the primary determinant of delay is the computing capacity rather than the energy budget.

VII. Conclusion

In this paper, we examine the joint optimization problem in a multi-UAV assisted MEC system, specifically focusing on service caching, offloading decision, and resource allocation. We formulate computational offloading and service caching optimization as a dual timescale problem, considering that frequent updates to the service caching cause time delay and energy consumption. The optimization objective is to minimize the total system latency while maintaining energy limitations for all UAVs and devices. Firstly, a service caching policy (SCP) based on task popularity and the GDSF algorithm is proposed, which outputs the caching decision over a longer time period. In contrast, the computation offloading decision and resource allocation proportions are determined over a shorter time period. Then, to solve this optimization problem, a dual timescale twin delayed deep deterministic policy gradient (DTTD3) algorithm is presented by incorporating the SCP into the TD3 algorithm, where the SCP is one of the states in the state space, and computational offloading and resource allocation decisions are obtained based on the current service caching state. The performance of DTTD3 algorithm is demonstrated to be superior to that of benchmarking schemes through simulation results.

References

[1] Y. Li, K. Sun, and L. Cai, "Cooperative device-to-device communication with network coding for machine type communication devices," *IEEE Trans. Wireless Commun.*, vol. 17, no. 1, pp. 296–309, Jan. 2018.

[2] H. Zhou, Z. Zhang, D. Li, and Z. Su, "Joint optimization of computing offloading and service caching in edge computing-based smart grid," *IEEE Trans. Cloud Comput.*, vol. 11, no. 2, pp. 1122–1132, Apr.–Jun. 2023.

[3] R. Tallat et al., "Navigating industry 5.0: A survey of key enabling technologies, trends, challenges, and opportunities," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1080–1126, 2nd Quart., 2023.

[4] H. Guo, X. Zhou, Y. Wang, and J. Liu, "Achieve load balancing in multi-UAV edge computing IoT networks: A dynamic entry and exit mechanism," *IEEE Internet Things J.*, vol. 9, no. 19, pp. 18725–18736, Oct. 2022.

[5] Y. Huang, Q. Wu, R. Lu, X. Peng, and R. Zhang, "Massive MIMO for cellular-connected UAV: Challenges and promising solutions," *IEEE Commun. Mag.*, vol. 59, no. 2, pp. 84–90, Feb. 2021.

[6] A. Al-Hourani, S. Kandeepan, and S. Lardner, "Optimal lap altitude for maximum coverage," *IEEE Wireless Commun. Lett.*, vol. 3, no. 6, pp. 569–572, Dec. 2014.

[7] K. Yu, K. Jin, and X. Deng, "Review of deep reinforcement learning," in *Proc. IEEE 5th Adv. Inf. Manag., Commun., Electron. Autom. Control Conf. (IMCEC)*, vol. 5, 2022, pp. 41–48.

[8] L. Zhang, B. Jabbari, and N. Ansari, "Deep reinforcement learning driven UAV-assisted edge computing," *IEEE Internet Things J.*, vol. 9, no. 24, pp. 25449–25459, Dec. 2022.

[9] Q. Luo, T. H. Luan, W. Shi, and P. Fan, "Deep reinforcement learning based computation offloading and trajectory planning for multi-UAV cooperative target search," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 504–520, Feb. 2023.

[10] X. Wei, L. Cai, N. Wei, P. Zou, J. Zhang, and S. Subramaniam, "Joint UAV trajectory planning, DAG task scheduling, and service function deployment based on DRL in UAV-empowered edge computing," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12826–12838, Jul. 2023.

[11] B. Dai, J. Niu, T. Ren, Z. Hu, and M. Atiquzzaman, "Towards energy-efficient scheduling of UAV and base station hybrid enabled mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 915–930, Jan. 2022.

[12] L. Bao, Z. He, J. Tan, Y. Chen, and M. Zhao, "Thermal-aware task scheduling and resource allocation for UAV-and-basestation hybrid-enabled MEC networks," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 579–593, Jun. 2023.

[13] Z. Ning et al., "Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2628–2644, May 2023.

[14] S. Zhu, L. Gui, D. Zhao, N. Cheng, Q. Zhang, and X. Lang, "Learning-based computation offloading approaches in UAVs-assisted edge computing," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 928–944, Jan. 2021.

[15] T. Zhang, Z. Wang, Y. Liu, W. Xu, and A. Nallanathan, "Joint resource, deployment, and caching optimization for ar applications in dynamic UAV noma networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 5, pp. 3409–3422, May 2022.

[16] J. Ji, K. Zhu, D. Niyato, and R. Wang, "Joint cache placement, flight trajectory, and transmission power optimization for multi-UAV assisted wireless networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5389–5403, Aug. 2020.

[17] J. Zhang, Y. Shen, Y. Wang, X. Zhang, and J. Wang, "Dual-timescale resource allocation for collaborative service caching and computation offloading in IoT systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 2, pp. 1735–1746, Feb. 2023.

[18] S. Bi, L. Huang, and Y.-J. A. Zhang, "Joint optimization of service caching placement and computation offloading in mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 19, no. 7, pp. 4947–4963, Jul. 2020.

[19] L. Chen, G. Gong, K. Jiang, H. Zhou, and R. Chen, "DDPG-based computation offloading and service caching in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2022, pp. 1–6.

[20] W. Wen, Y. Cui, T. Q. S. Quek, F.-C. Zheng, and S. Jin, "Joint optimal software caching, computation offloading and communications resource allocation for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 7, pp. 7879–7894, Jul. 2020.

[21] X. Ma, A. Zhou, S. Zhang, and S. Wang, "Cooperative service caching and workload scheduling in mobile edge computing," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2076–2085.

[22] Q. Shen, B.-J. Hu, and E. Xia, "Dependency-aware task offloading and service caching in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 12, pp. 13182–13197, Dec. 2022.

[23] G. Zheng, C. Xu, M. Wen, and X. Zhao, "Service caching based aerial cooperative computing and resource allocation in multi-UAV enabled MEC systems," *IEEE Trans. Veh. Technol.*, vol. 71, no. 10, pp. 10934–10947, Oct. 2022.

[24] Z. Yang, C. Pan, K. Wang, and M. Shikh-Bahaei, "Energy efficient resource allocation in UAV-enabled mobile edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 9, pp. 4576–4589, Sep. 2019.

[25] F. Zhou, Y. Wu, R. Q. Hu, and Y. Qian, "Computation rate maximization in UAV-enabled wireless-powered mobile-edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 1927–1941, Sep. 2018.

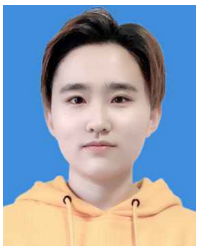
[26] Y. Liu, K. Xiong, Q. Ni, P. Fan, and K. B. Letaief, "UAV-assisted wireless powered cooperative mobile edge computing: Joint offloading, cpu control, and trajectory optimization," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 2777–2790, Apr. 2020.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.

[27] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1628–1656, 3rd Quart., 2017.
[28] Y. Al-Eryani, M. Akrout, and E. Hossain, "Multiple access in cell-free networks: Outage performance, dynamic clustering, and deep reinforcement learning-based design," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 4, pp. 1028–1042, Apr. 2021.
[29] N. Cheng et al., "Space/aerial-assisted computing offloading for IoT applications: A learning-based approach," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 5, pp. 1117–1129, May 2019.
[30] X. Zhou et al., "Edge computation offloading with content caching in 6G-enabled IoV," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 3, pp. 2733–2747, Mar. 2024.
[31] H. Zhou, Z. Zhang, Y. Wu, M. Dong, and V. C. M. Leung, "Energy efficient joint computation offloading and service caching for mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 2, pp. 950–961, Jun. 2023.



Na Lin received the M.S. degree in computer science from the Shenyang University of Technology in 2001, and the Ph.D. degree in computer science from Northeastern University, China, in 2005. As a Postdoctoral Researcher with Northeastern University from 2006 to 2010, she contributed to her field. She is Currently a Professor with the School of Computer Science, Shenyang Aerospace University, China. She was a Visiting Scholar with the University of Leicester, U.K., from 2019 to 2020. Her research interests encompass air-ground integrated networks, mobile edge computing, intelligent transportation, FANET, UAV swarm, and SDVN.



Xiao Han received the B.S. degree in electronic information engineering from Shenyang Aerospace University, Shenyang, China, in 2019, where she is currently pursuing the M.S. degree. Her research interests mainly include MEC, DRL, and UAV networks.



Ammar Hawbani (Member, IEEE) received the B.S. degree in computer software and theory and the M.S. and Ph.D. degrees from the University of Science and Technology of China (USTC) in 2009, 2012, and 2016, respectively. From 2019 to 2023, he served as a Postdoctoral Researcher with USTC until 2019, where he was an Associate Researcher with the School of Computer Science and Technology. He is currently a Full Professor with the School of Computer Science, Shenyang Aerospace University, with research expertise in IoT, WSNs, WBANs, WMNs, VANETs, and SDN.



Yunhe Sun (Student Member, IEEE) received the B.S. degree in electrical engineering and the Ph.D. degree in power electronics and power drives from Northeastern University, Shenyang, China, in 2016 and 2023, respectively. She currently serves as a Lecturer with the School of Computer Science, Shenyang Aerospace University, Shenyang. Her research focuses on power system resilience, security, catastrophe risk management, and structural controllability analysis of complex networks.



Yunchong Guan received the B.S. degree in computer science and technology from Dalian Maritime University in 2011, and the M.S. and Ph.D. degrees in computer application technology from Northeastern University, Shenyang, China, in 2013 and 2019, respectively. He is currently a Lecturer with the School of Computer Science, Shenyang Aerospace University, Shenyang. His research interests encompass space-air ground integrated networking and computing, intelligent transportation system, and wireless multimedia networking and communication for vehicles and UAVs.



Liang Zhao (Member, IEEE) received the Ph.D. degree from the School of Computing, Edinburgh Napier University in 2011. He is a Professor with Shenyang Aerospace University, China. He was an Associate Senior Researcher with Hitachi (China) Research and Development Corporation from 2012 to 2014. With 150+ publications, his research focuses on ITS, VANET, WMN, and SDN. He received the Best/Outstanding Paper Awards at 2015 IEEE IUCC, 2020 IEEE ISPA, 2022 IEEE EUC, and 2013 ACM MoMM. He was recognized in the Top 2% of scientists globally by Stanford University in 2022. He has chaired conferences like 2022 IEEE BigDataSE and 2021 IEEE TrustCom. He is serving as an Associate Editor for *Frontiers in Communications and Networking* and *Journal of Circuits, Systems, and Computers*. He is a JSPS Invitational Fellow in 2023.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 12:03:05 UTC from IEEE Xplore. Restrictions apply.